

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка
Отчет

Практическая работа

"ДЗ 2: Проектирование и технический дизайн API"

Выполнил:

Фёдоров Иван

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

- Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
- Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
- Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
- Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

Функциональные требования:

№	Требование
F1	Регистрация нового пользователя (email, пароль, ФИО, телефон)
F2	Аутентификация (вход по email/паролю) + JWT
F3	Получение / редактирование профиля пользователя
F4	Просмотр списка объектов недвижимости с пагинацией и фильтрацией (по типу, цене, городу, кол-ву комнат)
F5	Просмотр карточки объекта (фото, описание, условия аренды, данные арендодателя)
F6	Добавление объекта недвижимости (только для роли landlord)
F7	Редактирование / снятие с публикации своего объекта
F8	Добавление фото к объекту
F9	Отправка заявки на бронирование объекта (указать даты, сообщение)
F10	Просмотр своих бронирований (для tenant — поданные, для landlord — входящие)
F11	Подтверждение / отмена бронирования (landlord)

F12	Заключение договора после подтверждения брони (генерация contract
F13	Отправка сообщений между пользователями по объекту
F14	Просмотр истории сообщений по диалогу
F15	Добавление объекта в избранное / удаление
F16	Просмотр избранных объектов
F17	Оставление отзыва по завершённом договору
F18	История сделок (договоры и платежи) для пользователя

Нефункциональные требования:

№	Требование
N1	Время ответа API: < 200 мс (p95) для чтения, < 500 мс для записи
N2	Поддержка не менее 1000 RPS на чтение
N3	JWT-токены с access (15 мин) и refresh (7 дней)
N4	Все эндпоинты, кроме /auth/*, требуют валидный access token
N5	Ролевой доступ: landlord может создавать/редактировать объекты, tenant — только бронировать
N6	Пагинация всех списков (limit/offset или cursor)
N7	Валидация входных данных (например, даты: end_date >= start_date)
N8	Логирование всех действий с изменением данных
N9	Ограничение загрузки фото: макс 10 шт на объект, макс 5 MB на файл
N10	API версионизируется через префикс /api/v1

Выбор стиля API

Выбран: REST API

Обоснование:

1. CRUD-сущности хорошо ложатся на ресурсы
2. Есть стандартизированные HTTP-методы и коды ответов
3. Независимость от фронтенд-фреймворка
4. Легко документировать через OpenAPI

1. GET /health — проверка работоспособности

Тело запроса: отсутствует

Тело ответа (200 OK):

- status (string) — всегда "healthy"

Ошибки: отсутствуют

2. POST /auth/register — регистрация пользователя

Тело запроса: email, пароль, полное имя, телефон, опционально роль (tenant/landlord)

Тело ответа (201): user_id, email, full_name, phone, role

Ошибки: 400 (ошибка валидации), 409 (email уже существует)

3. POST /auth/login — вход в систему

Тело запроса: email, пароль

Тело ответа (200): access_token, refresh_token, token_type

Ошибки: 401 (неверные учетные данные)

4. GET /users/me — получить текущего пользователя

Тело запроса: отсутствует (требуется токен)

Тело ответа (200): user_id, email, full_name, phone, avatar_url, role

Ошибки: 401 (не авторизован)

5. GET /properties — поиск недвижимости

Тело запроса: отсутствует (параметры в query: city, property_type, min_price, max_price, rooms_count, limit, offset)

Тело ответа (200): total (количество) и items (массив объектов недвижимости)

Ошибки: отсутствуют

6. POST /properties — создать объект (только landlord)

Тело запроса: title, description, property_type, price_per_month, city, address, опционально security_deposit, rooms_count, area_sqm

Тело ответа (201): полный объект созданной недвижимости

Ошибки: 400, 401, 403 (нет роли landlord)

7. GET /properties/{property_id} — получить объект по ID

Тело запроса: отсутствует

Тело ответа (200): полный объект недвижимости с фото и условиями аренды

Ошибки: 404 (не найден)

8. GET /favorites — избранные объекты

Тело запроса: отсутствует (требуется токен)

Тело ответа (200): массив объектов недвижимости

Ошибки: 401

9. POST /favorites — добавить в избранное

Тело запроса: property_id

Тело ответа (201): пустое тело

Ошибки: 400, 401, 404

10. GET /bookings — мои бронирования

Тело запроса: отсутствует (опционально параметр status в query)

Тело ответа (200): массив бронирований (booking_id, property_id, start_date, end_date, status и др.)

Ошибки: 401

11. POST /bookings — создать бронирование (только tenant)

Тело запроса: property_id, start_date, end_date, опционально message

Тело ответа (201): полный объект бронирования

Ошибки: 400, 401, 403, 404, 409 (даты заняты)

12. PATCH /bookings/{booking_id}/status — изменить статус (только landlord)

Тело запроса: status (confirmed или cancelled)

Тело ответа (200): обновлённый объект бронирования

Ошибки: 400, 401, 403, 404

13. GET /messages — список диалогов

Тело запроса: отсутствует (требуется токен)

Тело ответа (200): массив диалогов с последним сообщением по каждому

Ошибки: 401

14. POST /messages — отправить сообщение

Тело запроса: receiver_id, property_id, message_text

Тело ответа (201): message_id, sender_id, receiver_id, property_id, message_text, sent_at, is_read

Ошибки: 400, 401, 404

15. GET /messages/dialogs/{user_id}/{property_id} — история переписки

Тело запроса: отсутствует

Тело ответа (200): массив сообщений (как в POST /messages)

Ошибки: 401, 403, 404

16. GET /contracts — история сделок

Тело запроса: отсутствует (требуется токен)

Тело ответа (200): массив договоров (contract_id, contract_number, start_date, end_date, monthly_rent, status и др.)

Ошибки: 401

Вывод

В ходе выполнения работы были описаны функциональные и нефункциональные требования к проекту, что позволит обеспечить ясность при разработке. Был выбран подход REST API. Также была разработана и реализована подробная OpenAPI-схема, включающая 16 эндпоинтов, с описанием формата запросов, ожидаемых данных, а также возможных форматов ответов и кодов ошибок. Спецификация документирована в Swagger UI и полностью покрывает функциональные требования сервиса аренды недвижимости.